

Naive Bayes

2026-04-17

Introduction

Naive Bayes is the simplest probabilistic classifier and one of the oldest. It applies Bayes' theorem under the strong assumption that features are conditionally independent given the class label. The independence assumption is almost always wrong in practice, yet Naive Bayes performs surprisingly well — often competitively with more sophisticated methods — on text classification, spam filtering, and other high-dimensional sparse problems. It trains in linear time and serves as a strong baseline for any classification pipeline.

Prerequisites

A working understanding of Bayes' theorem, conditional probability, and standard distributions (Gaussian, multinomial, Bernoulli) used as per-feature class-conditional models.

Theory

The classifier is

$$\hat{y}(x) = \arg \max_y P(y) \prod_{j=1}^P P(x_j | y),$$

where $P(y)$ is the class prior (estimated from class frequencies) and $P(x_j | y)$ is the per-feature class-conditional density. Three standard variants:

- **Gaussian Naive Bayes:** $P(x_j | y)$ is Normal with class-conditional mean and variance, suitable for continuous features.
- **Multinomial Naive Bayes:** for word counts and text classification.
- **Bernoulli Naive Bayes:** for binary features.

Training reduces to computing class frequencies and per-feature class-conditional statistics — extremely fast even on millions of examples.

Assumptions

Features are conditionally independent given the class (almost always violated but the classifier remains useful); the chosen distribution family matches each feature's behaviour.

R Implementation

```
library(e1071)

set.seed(2026)
n <- 400
df <- data.frame(
  x1 = rnorm(n), x2 = rnorm(n),
  x3 = sample(c("A", "B"), n, replace = TRUE),
```

```

y = factor(sample(c("pos", "neg"), n, replace = TRUE))
)

# Inject a relationship
df$y <- factor(ifelse(df$x1 + 0.5 * df$x2 +
                      as.numeric(df$x3) * 0.5 > runif(n, -1, 1),
                      "pos", "neg"))

idx <- sample(n, n * 0.7)
nb <- naiveBayes(y ~ ., data = df[idx, ])
pred <- predict(nb, df[-idx, ])

mean(pred == df[-idx, "y"])
nb$tables$x1           # Gaussian parameters for x1 per class

```

Output & Results

`naiveBayes()` returns a fitted classifier with per-class priors and per-feature conditional distribution parameters. `predict()` provides class labels and (with `type = "raw"`) class probabilities. The accuracy on a held-out fold gives a baseline against which more complex models can be compared.

Interpretation

A reporting sentence: “Gaussian Naive Bayes (with Laplace smoothing for the categorical predictor) achieved 74 % test accuracy, competitive with logistic regression’s 76 % at a fraction of the training cost; the per-class feature tables identify x_1 as the most discriminative feature (class-conditional mean difference 0.8).” Always state the variant used and the smoothing parameter for categorical features.

Practical Tips

- Naive Bayes is competitive on high-dimensional sparse data (text classification, spam filtering); often wins at small training sizes where complex models over-fit.
- Apply Laplace smoothing (`laplace = 1`) for categorical features to avoid zero-probability cases that would otherwise propagate to make the entire posterior zero.
- Probability calibration is often poor (the independence assumption produces overconfident predictions); apply Platt scaling or isotonic regression if calibrated probabilities are needed.
- For continuous features that are not Gaussian, transform (log, Box-Cox) or bin to ordinal before fitting; alternatively, use a kernel-density NB variant.
- The `naivebayes` package provides faster implementations and supports kernel density estimation for non-Gaussian continuous features.
- For text classification specifically, multinomial NB on TF-IDF features remains a strong baseline; only neural methods reliably beat it on large enough corpora.

R Packages Used

`e1071::naiveBayes()` for the classical implementation; `naivebayes` for faster alternatives with kernel-density and Poisson variants; `klaR::NaiveBayes()` for an interface with cross-validation hooks; `parSNIP::naive_Bayes()` for tidymodels integration; `text2vec` and `quanteda` for text feature extraction before NB classification.

For Reviewers

What to look for in a paper using this method.

- Common misapplications.
- Diagnostics that should be reported but often aren't.
- Red flags in tables and figures.
- What to verify.
- What an adequate Methods paragraph must contain.

See also — labs in this chapter

- Cross-validation, nested CV, bootstrap .632+
- Regularisation: ridge, lasso, elastic net
- Trees, random forests, gradient boosting
- Interpretability and SHAP
- Tabular neural networks with torch
- Imaging and sequence models intro
- tidymodels pipelines
- FDR, knockoffs, replication crisis